

Government Residence Women Polytechnic, Tasgaon



2026– 27

A report on

“INTERNSHIP”

(ITR – 315004)

Department of Computer Engineering

MAHARASHTRA STATE BOARD OF TECHNICAL EDUCATION, MUMBAI



CERTIFICATE

This is to certify that – students mentioned below of Diploma in Computer Engineering of Government Residence Women Polytechnic, Tasgaon (1228); have satisfactorily completed 12 -week duration internship and have submitted the report as partial fulfilment of the prescribed curriculum of **Maharashtra State Board of Technical Education Mumbai** for the academic year 2026- 27.

Roll. No.	Enrolment No.	Name of student
30	24210280163	Amruta Chandrakant Shinde

Dr. Vinod R. Falmari
Mentor

Smt. S. B. Patil
Head of Department
Computer Engineering

Dr. S. B. Patil
Principal
GRWPT

INDUSTRIAL CERTIFICATE:



ABSTRACT

The 12-week industrial training at iGAP technologies provided an excellent opportunity to bridge the gap between academic learning and industrial practices. The training was designed to enhance technical skills, professional competence, and practical exposure to real-world IT projects. During the program, I gained hands-on experience in areas such as software development, data science, artificial intelligence, and machine learning using Python. The training emphasized key concepts including data preprocessing, exploratory data analysis, implementation of machine learning models, and deployment of solutions for real-time applications.

Industrial Training provides required professional and practical skills to the students. So, it is very important for the student. This is an essential part in the development of the practical and professional skills required of an engineer and an aid to perspective employment. For this training, I am joining company iGAP Technologies PVT.Ltd There we learnt Data Science and AIML using Python and prepare project for it.

The industrial training enables students to go out and gain more practical knowledge of what they have been taught in school and to acquire industry bases skills.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to iGAP Technologies for providing me with the opportunity to undergo 12 weeks of industrial training in Data Science and Artificial Intelligence/Machine Learning using Python. This training has been an invaluable learning experience, allowing me to apply theoretical knowledge to practical, real-world problems and gain industry-relevant skills.

It is great privilege for me express our sincere thanks to Dr. S. B. Patil in-charge principal of Government Residence Women Polytechnic, Tasgaon for his valuable suggestions, and constant encouragement for our project work.

We sincerely acknowledge the help and co-operation from teaching and non-teaching staff of the Department of Computer Engineering, Government Residence Women's Polytechnic, Tasgaon.

Roll. No.	Enrolment No.	Name of student
30	24210280163	Amruta Chandrakant Shinde

Place: Tasgaon

Date:

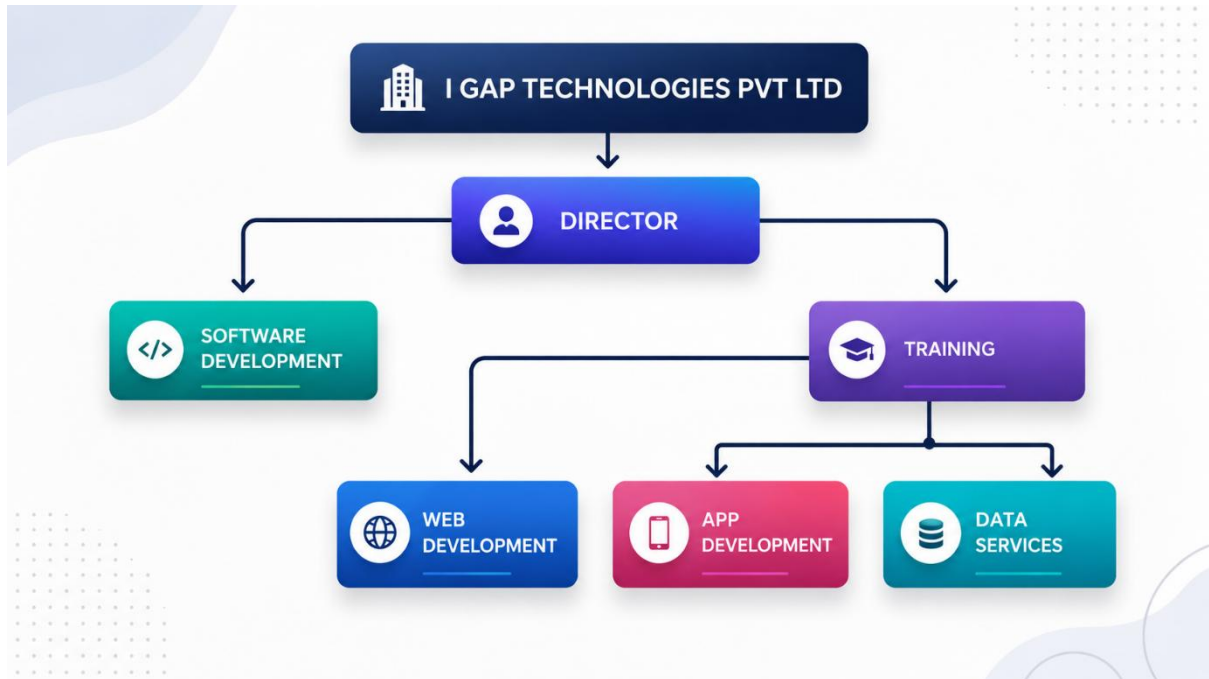
INDEX

Chapter.No.	Chapter Name	Page no.
Chapter 1:	Organisational Structure (Industry)	1-3
Chapter 2:	Introduction of Industry	4-9
Chapter 3:	Safety procedures followed and safety gears used by industry	10-11
Chapter 4:	Problem statement and requirement analysis	12-14
Chapter 5:	Proposed methodology and planning	15-18
Chapter 6:	Design: Flowcharts/ use case diagrams/ screen design/ DFD's	19-20
Chapter 7:	Implementation and testing	21-23
Chapter 8:	Challenges faced and future scope	24-25
Chapter 9:	Conclusion	26
Chapter 10:	Reference and Bibliography	27-28

CHAPTER 1

Organisational Structure (Industry)

Industry Name : iGAP TECHNOLOGIES PVT LTD



CEO (Chief Executive Officer)

- Overall head of the company.
- Defines company vision, strategy, and goals.
- Ensures all departments align with business objectives.
- Makes key decisions, manages stakeholders, and represents the company externally.

Chief Operating Officer (COO)

- Oversees day-to-day operations.
- Coordinates across departments to ensure smooth execution
- Focuses on operational efficiency, performance, and productivity.

Legal Team

- Handles legal compliance, contracts, intellectual property, and risk management.
- Advises on business deals and employment law.
- Protects the company against lawsuits and regulatory issues.

HR Head

- Manages recruitment, training, employee engagement, payroll, and performance.
- Builds policies to support workplace culture.
- Ensures compliance with labour laws.

Chief Financial Officer (CFO)

- Manages finances, budgets, investments, and financial risks.
- Prepares reports for decision-making.
- Ensures profitability and financial sustainability.

Operations Leader

- Ensures IT infrastructure, systems, and processes are reliable.
- Manages IT support, networking, and installations.
- Keeps operations running efficiently across all departments.

Chief Information Officer (CIO)

- Responsible for technology strategy and innovation.
- Oversees software development, IT systems, and digital transformation.
- Ensures the company stays competitive with new tech.

Under HR Head

- HR: Core employee management, benefits, training.
- Customer Relations: Builds client relationships, resolves customer issues.
- Admin: Handles office administration, facilities, support services.

Under CFO

- Accounts: Manages bookkeeping, invoices, payroll, taxes.
- Records: Maintains financial documentation.
- Corporate Services: Internal corporate management functions.
- Business Development: Identifies new business opportunities, partnerships.
- Marketing: Promotes company services, branding, and campaigns.
- Public Sector: Focuses on government/education clients.
- Private Sector: Focuses on corporate/enterprise clients.

- Customer Relations: Bridges marketing and client engagement.

Under Operations Leader

- Systems Support: Provides technical help for internal systems.
- Help Desk: First-level IT support for employees and clients.
- Installations & Networking: Sets up and maintains network infrastructure.
- IT/Ops: Oversees IT infrastructure and operations.
- Software Ops: Ensures software systems run smoothly.
- Comm & Systems: Manages communication systems (email, VoIP, etc.).
- IT Department: Core IT services across the organization.

Under CIO

- Analysis & Design: Plans and designs software/IT systems.
- Software Analysis: Requirement gathering and feasibility.
- Software Architecture: Defines technical structure of applications.
- Software Design: Creates detailed system and UI/UX designs.
- Programming: Develops software and applications.
- VB / C+ / Java: Different programming languages and teams

CHAPTER 2

Introduction Of Industry

iGAP Technologies Pvt Ltd is a dynamic software development and training company, providing cutting-edge solutions and comprehensive training programs. Established initially as a proprietary company in 2017, iGAP Technologies evolved into a Private Limited Company in September 2021. The company is spearheaded by two directors, Mr. Abhijit Prakash Gatade, a Computer Engineer, and Mr. Dilip Prakash Gatade, a Master in Business Administration. Together, they bring a wealth of knowledge and experience, having also managed multiple businesses in the healthcare and education sectors.



Departments & Services

iGAP Technologies offers a wide range of services tailored to meet the diverse needs of its clients. The primary service offerings include:

- **Web Application Development:** Crafting robust, scalable, and secure web applications that cater to various business needs.
- **Website Development:** Creating engaging and user-friendly websites that enhance online presence and user experience.
- **Mobile Application Development:** Developing intuitive and high-performing mobile applications using platforms like Flutter.
- **Data-related Services:** Providing data analysis, management, and engineering services to help businesses make data-driven decisions.

Additionally, iGAP Technologies is renowned for its comprehensive training programs in the following areas:

- **Full Stack Web Development:** Equipping students with the skills to build complete web solutions from front-end to back-end.
- **Flutter Mobile Application Development:** Training in creating cross-platform mobile applications using the Flutter framework.
- **Machine Learning Engineering:** Teaching the fundamentals and advanced concepts of machine learning to prepare students for careers in AI and data science.

History

The journey of iGAP Technologies began in 2017, founded as a proprietary company with a vision to deliver exceptional software solutions and training. Recognizing the growing demand and the need for a more structured organizational framework, the company transitioned into a Private Limited Company in September 2021. Over the years, iGAP Technologies has expanded its client base across India, offering top-notch service-based solutions in web, mobile, and data technologies. The leadership of Mr. Abhijit Prakash Gatade and Mr. Dilip Prakash Gatade has been instrumental in steering the company towards success, leveraging their expertise and experience in both IT and business management.

Processes

- **Project-Based Work:** The company works on live, task-based projects, allowing for hands-on experience.
- **Agile and Collaborative:** iGAP Technologies likely uses the software development life cycle, with teams working collaboratively on data-driven projects in a potentially agile environment.
- **Client-Focused Delivery:** Processes are centered on delivering customized, cost-effective, and long-term software solutions that meet client objectives.

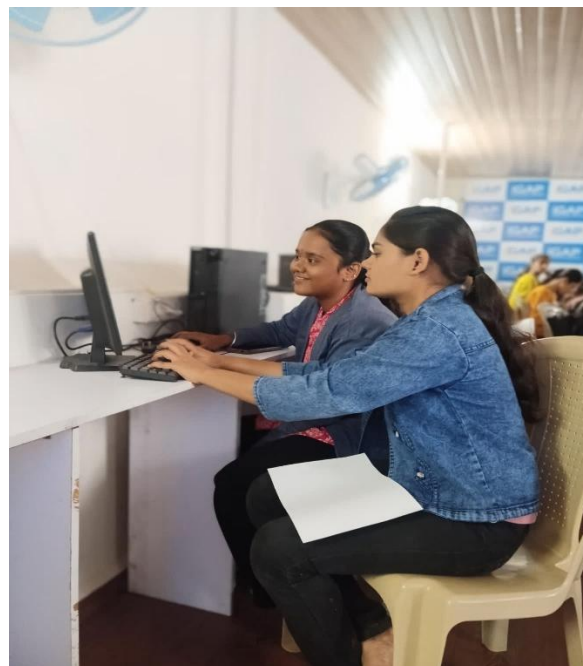
Work Culture

- **Collaborative & Supportive:** The culture is described as collaborative, with supportive teams and a focus on teamwork.
- **Skill Development:** There is a strong emphasis on skill development and providing employees with practical tools and techniques.
- **Work-Life Balance:** The company aims to promote a good work-life balance and overall well-being among its employees.
- **Smart Casual Dress Code:** Employees typically follow a smart-casual dress code that combines professionalism with comfort.

Potential Downsides Mentioned in Reviews

- **Infrastructure:** Some past reviews have noted that the infrastructure, such as PCs, was not always upgraded.
- **Internship Benefits:** A few reviews mentioned a lack of benefits after internships, with some also suggesting that the technology used for training could be improved.

Geotag photos:





Project Presentation



CHAPTER 3

Safety Procedures Followed and Safety Gears Used by Industry

To maintain a safe, secure, and professional working environment, iGAP Industry follows strict organizational rules and behavioural guidelines. These policies ensure employee safety, smooth operations, and a positive workplace culture.

1. Always follow the institution's and organization's social media usage policies.
2. Never disclose the names or personal details of students or clients in public forums.
3. Be cautious when sharing project or user-related information with teachers, colleagues, or outsiders.
4. Use only official and verified email addresses for communication (e.g., college or company email).
5. Understand and follow the guidelines of the Children's Online Privacy Protection Act (COPPA) if working with minors' data.
6. Choose tools, libraries, and technologies from secure, reputable sources only.
7. Do not share or expose sensitive data such as passwords, user information, or datasets.
8. Never arrange to meet anyone in person based on only online communication, without proper supervision.
9. Remain alert to cyberbullying or unethical online behavior—report anything suspicious.
10. Avoid posting or sharing anything that could hurt someone emotionally, culturally, or professionally.
11. Keep all your passwords secure—do not share them with friends, even close ones.
12. Practice ethical digital behavior—stay respectful, responsible, and professional Online.
13. Maintain academic honesty—do not plagiarize or fake data in reports or projects.
14. Take regular breaks during computer use to avoid eye strain and mental fatigue.
15. Always verify facts or information found online—don't rely blindly on any Source.

CHAPTER 4

Problem Statement and Requirement Analysis

Problem Statement

Students have to study from a large amount of educational material such as college notes, textbooks, lecture notes, and PDF documents. Finding a particular topic or answer from lengthy study material manually is time-consuming and difficult, specially during examination preparation.

Traditional PDF readers mainly allow users to read and search documents using keywords. They do not provide contextual answers, automatic summaries, practice MCQs, or important examination questions from the student's own study material. Students therefore need to use different tools for different study activities.

There is therefore a need for an intelligent system that can understand uploaded college study material, retrieve relevant information, and provide meaningful answers to students in natural language.

The **AI College Assistant** solve this problem by providing an AI-powered platform where students can upload college PDFs and interact with their study material using natural language. The system extracts text from the PDF, divides the content into smaller chunks, generates semantic embeddings, stores them in a FAISS vector database, and uses Retrieval-Augmented Generation (RAG) to retrieve relevant information.

The retrieved information is provided to the Gemini AI model to generate answers based on the uploaded document. The system also provides study summaries, MCQ generation, important examination questions, and chat history.

In short, the problem is to design and develop an AI-based educational assistant that can reduce the time required to search and understand college study material and provide multiple study-support features through a single platform.

Requirement Analysis

1. Functional Requirements

1. **PDF Upload** – The system must allow students to upload college study material in PDF format.
2. **PDF Text Extraction** – The system should extract text from the uploaded PDF for further processing.
3. **Text Chunking** – The extracted document should be divided into smaller overlapping chunks to improve information retrieval.
4. **Text Embedding** – The system should convert document chunks into numerical vector representations using a Hugging-Face sentence-transformer model.
5. **Vector Storage** – The generated embeddings should be stored in a FAISS vector database.
6. **Similarity Search** – The system should retrieve relevant document chunks according to the user's question.

7. **AI Question Answering** – The system must generate answers using the retrieved document information and Gemini AI.
8. **Study Summary** – The system should generate concise study notes from uploaded study material.
9. **MCQ Generation** – The system should generate multiple-choice questions based on the study material.
10. **Important Questions** – The system should generate important questions useful for examination preparation.
11. **Chat History** – The system should store previous questions and AI-generated answers.
12. **History Management** – The user should be able to view and clear previous chat history.
13. **Error Handling** – The system should display appropriate messages when a PDF is missing, text cannot be extracted, or the AI API is unavailable.
14. **User Interface** – The system should provide a simple and interactive web interface for accessing all features.
15. **Study Planner**-The user should plan study hours according to their time with the help of AI.
16. **Progress Bar**-The task assign to that day should be completed by user then progress bar will show that.

2. Non-Functional Requirements

- **Accuracy** – The system should provide answers relevant to the uploaded study material.
- **Performance** – Document retrieval and AI response generation should be completed within a reasonable time.
- **Reliability** – The application should handle invalid documents, missing API keys, and API errors properly.
- **Usability** – The interface should be simple and easy for students to operate.
- **Scalability** – Additional educational features and AI models should be capable of being added in the future.
- **Maintainability** – The project should be divided into separate modules for easier maintenance.
- **Security** – API keys must be stored securely and should not be exposed in the source code.
- **Portability** – The application should be capable of running on standard computers with Python and the required libraries.

3. Hardware Requirements

- Laptop/Desktop Computer
- Minimum 8 GB RAM
- Minimum 10 GB available storage
- Internet connection for Gemini API
- Keyboard and Mouse
- Display Monitor

4. Software Requirements

Programming Language:

Python

Framework:

Streamlit

Libraries/Packages:

- PyMuPDF – PDF text extraction
- LangChain – RAG and document processing
- LangChain HuggingFace – embedding integration
- HuggingFace Sentence Transformers – semantic embeddings
- FAISS – vector database and similarity search
- Google GenAI – Gemini API integration
- SQLite – chat history storage
- Streamlit Option Menu – navigation interface

Operating System:

Windows

Development Environment:

VS Code / Python IDE

5. User Requirements

- The user should be able to upload a college PDF easily.
 - The user should be able to ask questions in natural language.
 - The user should receive answers based on the uploaded document.
 - The user should be able to generate study summaries.
 - The user should be able to generate MCQs.
 - The user should be able to generate important examination questions.
 - The user should be able to view previous conversations.
 - The user should be able to clear chat history when required.
 - The system should display clear error messages when an operation cannot be completed.
-

CHAPTER 5

Project Planning

The project was planned and developed in different phases to ensure systematic development of the **AI College Assistant**.

Week 1: Research and Project Selection

- Study problems faced by students while using lengthy study materials.
- Research existing AI-based educational applications.
- Select the project topic and check its feasibility.

Week 2: Project Finalisation

- Finalize the **AI College Assistant** project.
- Define project objectives, scope, features, and expected outcomes.

Week 3: Project Planning

- Divide the project into different modules.
- Plan PDF processing, RAG, AI features, vector database, and chat history.

Week 4: Synopsis Preparation

- Prepare the project introduction, problem statement, objectives, scope, technologies, and expected outcomes.
- Present the project synopsis.

Week 5: Requirement Gathering

- Identify functional and non-functional requirements.
- Finalize hardware, software, libraries, and AI requirements.

Week 6: Environment Setup

- Install Python, Streamlit, and required libraries.
- Configure the development environment and Gemini API.

Week 7: System Design

- Design the system architecture and workflow.
- Plan PDF processing, vector store, RAG, AI features, and database modules.

Week 8: Implementation

- Implement PDF extraction, text chunking, embeddings, FAISS, RAG, Gemini integration, and AI features.
- Integrate all modules into Streamlit.

Week 9: Testing

- Test PDF processing, AI Chat, Summary, MCQ Generator, Important Questions, and Chat History.
- Test error handling and API integration.

Week 10: Documentation

- Prepare system documentation, screenshots, testing results, challenges, and future scope.

Week 11: Presentation

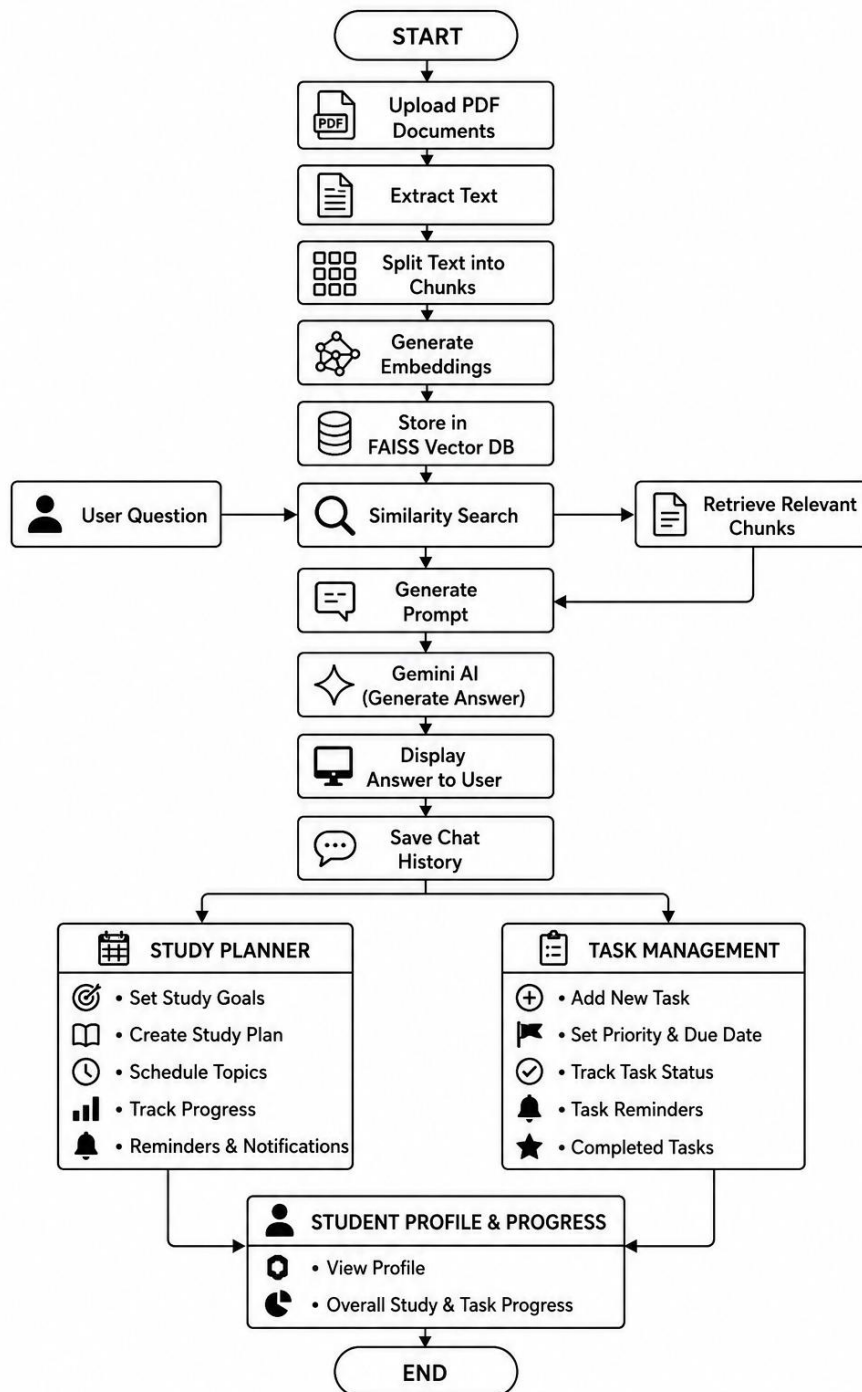
- Prepare the PPT and demonstrate the project features, architecture, technologies, and results.

Week 12: Final Submission

- Submit the final report, source code, presentation, and working application.
 - Demonstrate the completed **AI College Assistant**.
-

CHAPTER 6

Design: Flowchart



Flowchart Description

1. **Start**- The application is initialized using Streamlit.
2. **Upload PDF**- The user uploads college study material through the application.
3. **Extract Text**-The system extracts readable text from the uploaded PDF using PyMuPDF.

4. **Text Chunking**- The extracted text is divided into smaller overlapping chunks.
5. **Embedding Generation**- Each text chunk is converted into a vector representation using the HuggingFace sentence-transformer model.
6. **FAISS Vector Store**- The generated embeddings are stored in FAISS.
7. **Ask Question**- The user enters a question related to the uploaded study material.
8. **Similarity Search**- FAISS searches for the most relevant document chunks.
9. **Context Retrieval**- The retrieved chunks are combined as context.
10. **Gemini AI**- The context and user question are provided to Gemini through the RAG process.
11. **Generate Answer**- Gemini generates a natural-language answer based on the retrieved document information.
12. **Display Answer**- generated answer is displayed to the user.
13. **Save Chat History**- The question, answer, and timestamp are stored in the database.
14. **Study Planner**- Designs study plan for the task.
15. **Task Management**-Manages task as per schedule.
16. **End**- The user can ask another question or use another feature.

Use Case Example

Scenario:

A student wants to understand a topic from an uploaded college PDF.

User:

College Student

System:

AI College Assistant

Precondition:

A valid college PDF has been uploaded and processed.

Main Success Scenario:

- The student uploads a college PDF.
- The system extracts the text.
- The document is divided into chunks.
- Embeddings are generated and stored in FAISS.
- The student opens the AI Chat section.
- The student enters a question.
- FAISS retrieves relevant information from the document.
- The retrieved context is sent to Gemini.
- Gemini generates an answer.
- The answer is displayed to the student.
- The question and answer are saved in chat history.

Postcondition:

The student receives an AI-generated answer based on the uploaded study material and can continue asking questions.

CHAPTER 7

Implementation And Testing

Implementation Section

This section explains how the AI College Assistant was developed and integrated.

1. System Setup

Hardware Used:

- Laptop/Desktop
- Minimum 8 GB RAM
- Internet connection

Software Tools:

- Python
- Streamlit
- VS Code
- Git and GitHub

Libraries:

- PyMuPDF
- LangChain
- HuggingFace
- FAISS
- Google GenAI
- SQLite

2. PDF Processing

The user uploads a PDF through the Streamlit interface.

The PDF text is extracted using PyMuPDF. The extracted text is stored in the application's session state for further processing.

The system also checks whether the PDF contains readable text before creating the vector store.

3. Text Chunking

Large PDF documents are divided into smaller chunks using a recursive text splitter.

The implemented configuration uses:

- Chunk size: 1000 characters

- Chunk overlap: 200 characters

The overlap helps preserve contextual information between neighbouring chunks.

4. Feature / Embedding Generation

The system uses:

Model: sentence-transformers/all-MiniLM-L6-v2

The model converts each text chunk into a numerical vector representation.

These vector representations allow the system to perform semantic similarity search.

5. Vector Store

FAISS is used to store the generated embeddings.

When the user asks a question, FAISS performs similarity search and retrieves the most relevant chunks from the uploaded document.

The application retrieves the top relevant chunks and passes them to the RAG pipeline.

6. RAG and AI Model

The RAG system combines retrieval and generation.

The process is:

Question → FAISS Search → Relevant Document Chunks → Context → Gemini → Answer

The prompt instructs Gemini to use the retrieved college notes and avoid inventing information that is not present in the uploaded material.

7. AI Chat

The AI Chat module provides a text input where the student can enter questions.

The question is passed to the RAG module. Relevant information is retrieved from FAISS and provided to Gemini.

The generated answer is then displayed in the Streamlit interface.

8. Study Summary

The Study Summary module uses the uploaded PDF text and generates concise study notes.

The feature is intended to help students revise lengthy study material more efficiently.

9. MCQ Generator

The MCQ Generator uses the uploaded study material to generate practice multiple-choice questions.

The user can select the required number of questions and generate MCQs for examination preparation.

10. Important Questions

The Important Questions module analyzes the uploaded study material and generates questions that can be useful for examination preparation.

The user can select the required number of questions.

11. Chat History

SQLite is used to store:

- User question
- AI answer
- Date/time

The Chat History section displays previous conversations and provides an option to clear the stored history.

Testing Section

The testing process was performed to verify the functionality of each module.

1. Testing Strategy

- Unit testing of individual modules.
- Integration testing of PDF processing and vector retrieval.
- Testing of RAG question answering.
- Testing of AI summary generation.
- Testing of MCQ generation.
- Testing of important-question generation.
- Testing of database operations.
- Testing of Streamlit user interface.
- Testing of error handling.

2. Test Cases

Test Case	Input	Expected Result	Status
-----------	-------	-----------------	--------

PDF Upload	Valid PDF	PDF uploaded successfully	Pass
Text Extraction	Text-based PDF	Text extracted successfully	Pass
Empty PDF	PDF without readable text	Error message displayed	Pass
Chunking	Extracted PDF text	Text divided into chunks	Pass
FAISS Creation	Text chunks	Vector store created	Pass
AI Chat	Question from PDF	Relevant AI answer generated	Pass
Invalid Question	Empty question	Warning displayed	Pass
Summary	Uploaded study material	Summary generated	Pass*
MCQ	Uploaded study material	MCQs generated	Pass*
Important Questions	Uploaded study material	Questions generated	Pass*
Chat History	Question and answer	Conversation saved	Pass
Clear History	Existing history	History deleted	Pass

Note: AI-generation features depend on Gemini API availability and quota.

3. Results

The implemented system successfully performs the major functions required for an AI-based college study assistant.

The PDF processing and RAG pipeline were tested using college study material. The system successfully extracts the document content, creates embeddings, performs similarity search, and generates answers using retrieved information.

The Streamlit interface successfully integrates PDF upload, AI Chat, Study Summary, MCQ Generator, Important Questions, and Chat History into a single application.

4. Case Studies

Case Study 1 – AI Chat

A college PDF is uploaded and a question related to the content is entered. The system retrieves relevant chunks from FAISS and generates an answer through Gemini.

Case Study 2 – Study Summary

A lengthy study document is uploaded. The Summary module processes the material and generates concise revision notes.

Case Study 3 – MCQ Generation

The user selects the number of questions and the system generates practice MCQs based on the study material.

Case Study 4 – Important Questions

The system generates important questions from the uploaded document to support examination preparation.

Case Study 5 – Chat History

Questions and generated answers are stored in SQLite and displayed in the Chat History section.

5. Limitations & Challenges

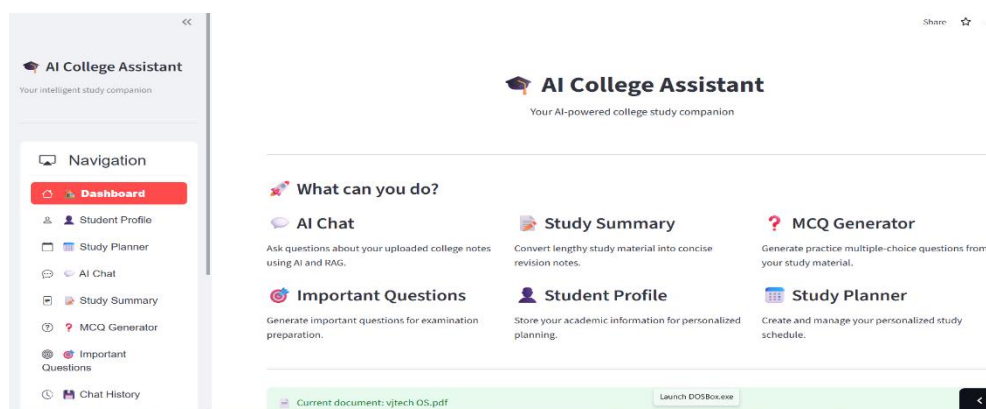
- Gemini API requests are subject to API quota limits.
- Internet connectivity is required for Gemini API requests.
- Scanned/image-only PDFs may not provide extractable text without OCR.
- Very large documents may require more processing time.
- AI-generated content depends on the quality of the retrieved document context.
- API errors or quota exhaustion can temporarily prevent AI generation.
- The system is primarily designed for document-based educational content.

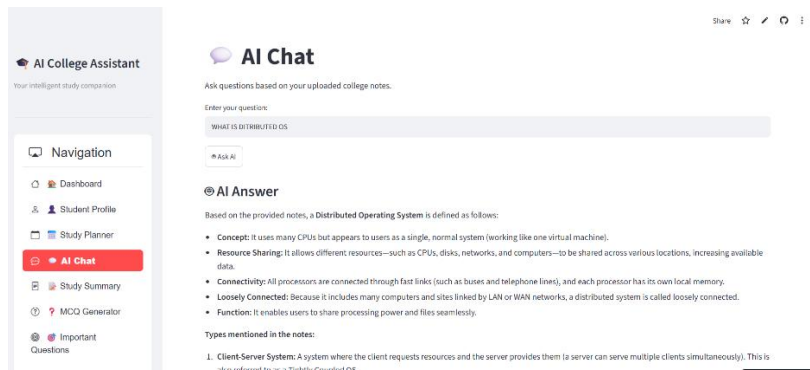
6. Improvements

- Add OCR support for scanned PDFs.
- Support multiple PDF documents simultaneously.
- Improve document retrieval using advanced embedding models.
- Add citation/page-number references in AI answers.
- Add user authentication.
- Add multilingual support.
- Add downloadable summaries and MCQs.
- Add a dashboard showing study activity.
- Add more advanced conversation memory.

Implementation = how the system was designed and developed using PDF processing, chunking, embeddings, FAISS, RAG, Gemini, Streamlit, and SQLite.

Testing = how each module was tested, including PDF processing, retrieval, AI generation, database operations, and error handling.





Smart Study Planner

Create and manage your personalized AI-powered study schedule.

Welcome Amruta Shinde!

Subjects

3

Daily Study Hours

5.0 hrs

Study Progress

Total Tasks

1

Completed

0

0% Completed

Today's Study Plan

Operating system

Topic: Process Management

01:00 - 03:00

1.0 hours

Priority: High

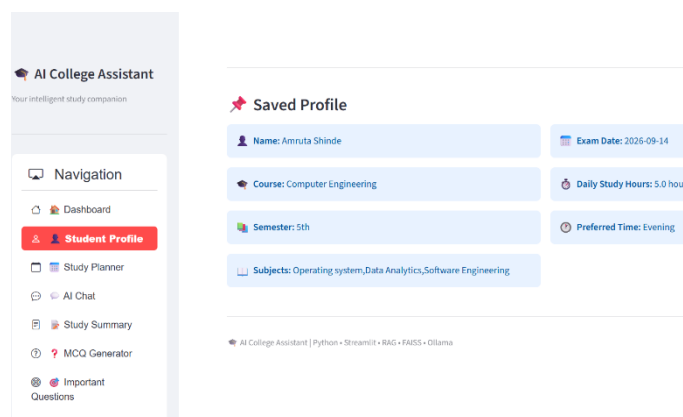
Complete

Delete

All Study Plans

2026-09-28 | Operating system | Process Management | 01:00-03:00 | High

Clear All Study Plans



Navigation

Dashboard

Student Profile

Study Planner

AI Chat

Study Summary

MCQ Generator

Important Questions

Chat History

Study Material

Upload College PDF

vjtech OS.pdf

1.8MB

Chat History

2026-08-28 03:47:07

Question:

WHAT IS DITRIBUTED OS

AI Answer:

Based on the provided college notes, a **Distributed Operating System** can be defined as follows:

- **Concept:** It is an operating system that uses many CPUs but appears to users as a single, normal operating system.
- **Resource Sharing:** It enables users to share processing power, files, and resources—such as CPU locations.
- **Connectivity:** It is a loosely connected system containing many computers and sites linked by LANs.
- **Processors:** All processors are connected through fast links (such as buses and telephone lines).

Clear Chat History

CHAPTER 8

Challenges Faced and Future Scope

Challenges Faced

a) Technical Challenges

- Integrating multiple AI and NLP libraries into a single application.
- Implementing semantic document retrieval using FAISS.
- Selecting appropriate chunk size and chunk overlap.
- Connecting the RAG pipeline with the Gemini API.
- Managing Streamlit session state for uploaded documents.
- Handling API errors and quota limitations.
- Deploying the application successfully on Streamlit Cloud.

b) Data-Related Challenges

- Extracting clean text from different PDF formats.
- Maintaining meaningful context while splitting documents.
- Handling large study documents.
- Ensuring that retrieved chunks are relevant to the student's question.
- Handling PDFs containing images or scanned pages.

c) System Challenges

- API quota limitations can affect AI-generated features.
- Internet connectivity is required for Gemini.
- Processing large PDFs can require additional time.
- Incorrect or incomplete document content can affect the generated response.
- Managing API secrets securely during local development and deployment.

Future Scope

a) Technical Improvements

- Add OCR support for scanned PDF documents.
- Use advanced embedding models for improved semantic retrieval.
- Implement hybrid keyword and semantic search.
- Add document re-ranking for more accurate retrieval.
- Add page-level source citations.
- Improve prompt engineering for more reliable answers.

b) System Enhancements

- Support multiple PDFs simultaneously.
- Allow students to organize documents by subject.
- Add user login and personalized study profiles.
- Add downloadable PDF/Word summaries.
- Add interactive MCQ tests with scores.

- Add automatic subject-wise important-question generation.
- Add voice-based question answering.
- Add multilingual question answering.

c) Broader Applications

The system can be adapted for:

- School education
- College education
- Competitive examination preparation
- Training and certification material
- E-learning platforms
- Institutional knowledge bases
- Technical documentation
- Research document assistance

The AI College Assistant can also be expanded into a complete AI-based learning platform that combines document understanding, personalized learning, assessment, and progress tracking.

CHAPTER 9

Conclusion

The **AI College Assistant** was developed to address the difficulties students face while studying from lengthy college notes and PDF documents. The system provides an interactive platform that allows students to upload study material and obtain useful information through Artificial Intelligence.

The project combines **PDF text extraction, text chunking, HuggingFace embeddings, FAISS vector search, Retrieval-Augmented Generation (RAG), Gemini AI, Streamlit, and SQLite** to create an integrated educational assistant.

The RAG-based AI Chat feature retrieves relevant information from the uploaded study material before generating an answer. This allows the system to provide document-based responses rather than relying only on general AI knowledge.

The project also provides additional educational features such as **Study Summary, MCQ Generator, Important Questions, and Chat History**. These features help students understand, revise, and practise their study material from a single application.

During implementation, challenges such as document processing, vector retrieval, API integration, API quota limitations, and deployment were addressed. Testing showed that the major application modules work together successfully.

Overall, the AI College Assistant demonstrates how Generative AI, NLP, semantic search, vector databases, and RAG can be combined to develop a practical educational application. The system provides a strong foundation for future development into a more personalized and comprehensive AI-powered learning platform.

CHAPTER 10

References and Bibliography

1. Python Software Foundation, **Python Documentation**.
[Python Documentation](#)
2. Streamlit, **Streamlit Documentation**.
[Streamlit Documentation](#)
3. LangChain, **LangChain Documentation**.
[LangChain Documentation](#)
4. Meta AI Research, **FAISS – Facebook AI Similarity Search**.
[FAISS Documentation](#)
5. Hugging Face, **Sentence Transformers Documentation**.
[Hugging Face Sentence Transformers](#)
6. Google, **Gemini API Documentation**.
[Gemini API Documentation](#)
7. Google, **Gemini API – Generate Content Documentation**.
[Gemini Generate Content Documentation](#)
8. PyMuPDF, **PyMuPDF Documentation**.
[PyMuPDF Documentation](#)
9. SQLite, **SQLite Documentation**.
[SQLite Documentation](#)
10. GitHub, **GitHub Documentation**.
[GitHub Documentation](#)
11. Lewis, P. et al., **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks**, 2020.
12. Reimers, N. and Gurevych, I., **Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks**, 2019.

